UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS OPERATIVOS

Grupo: 07

TRABAJO ESCRITO

Meltdown y Spectre

“Fallas críticas en la Seguridad de los Procesos Modernos”

ESTUDIANTES

Merida Serralde Francisco Jared y
Quezada Yépez Leonardo André

PROFESOR

Dr. Gunnar Eyal Wolf Iszaevich

30/04/2026

Índice

Índice **2**
Introducción **3**
Objetivos **3**
Desarrollo **4**
1. Conceptos clave para entender Meltdown y Spectre 4
1.1 Memoria Virtual. 4
1.2 Segmentación de instrucciones (pipeline) 5
1.2.1 Mecanismos de optimización de rendimiento 5
1.3 Topología de caché de la CPU 6
2. MELTDOWN y SPECTRE 7
2.1 MELTDOWN 7
2.1.2 Ejemplo de ejecución 9
2.1.3 Medidas de mitigación 10
2.2 SPECTRE 10
2.2.1 Plan de ataque 10
2.2.2 Ejemplo de ejecución 11
2.2.3 Medidas de mitigación 12
3 Conclusión final 12
Bibliografía **13**

Introducción

Típicamente, asociamos el concepto de explotación de vulnerabilidades a las relacionadas a nivel software, las cuales permiten a los atacantes acceder a información sensible, escalar privilegios, entre otros. Normalmente, este tipo de amenazas se pueden mitigar al actualizar el sistema operativo o de las propias aplicaciones. Sin embargo, esto cambió con la aparición de vulnerabilidades propias del diseño interno de los procesadores modernos.

Estos ataques se aprovechan de mecanismos pensados para la optimización del rendimiento (como puede ser la ejecución especulativa), y aprovechan vulnerabilidades a nivel hardware, afectando a una amplia variedad de arquitecturas, desde dispositivos móviles, hasta servicios en la nube.

Dentro de este ámbito, Meltdown y Spectre son las vulnerabilidades más difundidas a nivel global. Identificadas en procesadores de fabricantes como Intel, Apple, AMD y ARM, presentaron vulnerabilidades capaces de vulnerar el aislamiento de la memoria, y filtrar datos sensibles como contraseñas o claves de cifrado.

En este texto, hablaremos sobre las vulnerabilidades de Meltdown y Spectre, su funcionamiento, los mecanismos que explotaron y las soluciones y parches implementados para su control.

Objetivos

- Explicar la naturaleza de las vulnerabilidades Meltdown y Spectre, su impacto en los sistemas informáticos y las medidas implementadas para mitigarlas.

- Describir qué son Meltdown y Spectre y cómo funcionan a nivel de hardware.

- Analizar las causas del problema, enfocándose en los errores de diseño en los procesadores.

- Explicar los riesgos de seguridad asociados, como el acceso no autorizado a información sensible.

- Presentar las soluciones y parches desarrollados por los sistemas operativos (Windows, macOS y Linux).

Desarrollo

1. Conceptos clave para entender Meltdown y Spectre

A continuación exponemos brevemente algunos de los conceptos que conviene entender sobre arquitectura de procesadores para poder hablar correctamente sobre cómo funcionan las vulnerabilidades de Meltdown y Spectre. Cabe recalcar que parte de la información abordada hace referencia a la arquitectura utilizada antes de que se identificaran Meltdown y Spectre.

1.1 Memoria Virtual.

Los sistemas operativos modernos implementan la memoria virtual mediante un esquema de paginación que permite gestionar eficientemente la memoria en entornos multitarea y multiusuario.

En este modelo, el espacio de direcciones virtuales de cada proceso se divide en bloques de tamaño fijo llamados páginas, mientras que la memoria física se divide en bloques del mismo tamaño llamados marcos de página. El sistema operativo asigna dinámicamente marcos a las páginas de cada proceso guardando un mapeo en una tabla de páginas.

Cada proceso tiene su propio espacio de direccionamiento virtual, lo que da la ilusión de ejecutarse de forma aislada con acceso exclusivo a la memoria. Sin embargo, puesto que es imposible tener un sistema con suficiente memoria física como para mapear el espacio virtual de todos los procesos que se ejecutan, sólo las más utilizadas se almacenan en memoria principal.

Para acceder a estas páginas, se utiliza un mecanismo de traducción llamado Translation Lookaside Buffer (TLB), que es una caché que almacena traducciones recientes. Esto es importante tenerlo en cuenta pues tanto la traducción como los accesos a memoria dejan un rastro en la microarquitectura.

El principal causante de las vulnerabilidades vistas, en relación con la memoria virtual, es que por razones de eficiencia, el kernel está mapeado dentro del espacio virtual de cada proceso. Sin embargo, éstas no son accesibles en modo usuario y su acceso está protegido por bits de privilegio.

Si un proceso en modo usuario intenta acceder al modo kernel, se producirá un fallo de página, momento en el que el sistema operativo lo finaliza.

1.2 Segmentación de instrucciones (pipeline)

Los procesadores modernos presentan una alta complejidad que permite que la ejecución de instrucciones a nivel máquina no se limite a una mera ejecución en orden. Existen diversos mecanismos a nivel hardware que permiten acelerar la ejecución de estas instrucciones y aumentar el paralelismo.

En el contexto de Meltdown y Spectre, es necesario entender el funcionamiento de estos mecanismos para poder dar explicación a la explotación de las vulnerabilidades presentadas.

1.2.1 Mecanismos de optimización de rendimiento

Comenzando con el concepto de ejecución fuera de orden (out-of-order) es una técnica que permite maximizar el uso de todas las unidades de ejecución de un núcleo. En lugar de procesar las instrucciones únicamente en orden secuencial, la CPU las ejecuta tan pronto como todos los recursos necesarios estén disponibles. De forma que se puede ejecutar una instrucción antes de que el procesador haya confirmado los resultados de todas las instrucciones previas.

Sin embargo, estas instrucciones sólo pueden ser ejecutadas en paralelo siempre y cuando no exista una dependencia de datos entre ellas.

Para mejorar aún más el rendimiento, las CPUs implementan el concepto de **ejecución especulativa**, la cual consiste en ejecutar instrucciones antes de que se haya determinado si estas son necesarias y confirmadas. El caso más relevante de ejecución especulativa para este la evaluación de Meltdown y Spectre, especialmente este último, es el predictor de saltos.

En estas, se determina qué dirección tomará una bifurcación antes de que su condición sea evaluada. Las instrucciones que no tengan dependencia pueden ejecutarse en paralelo por adelantado. Si la predicción fue correcta, los resultados se confirman en orden, en caso contrario, el reorder buffer (ROB) permite volver a un estado consistente, limpiando y reiniciando las operaciones realizadas, sin embargo, no limpia el caché.

1.3 Topología de caché de la CPU

Como último concepto importante, hablamos sobre el papel de la memoria caché en Meltdown y Spectre.

Las CPU modernas incorporan una jerarquía de memoria caché diseñada para reducir la latencia de acceso a datos. Esta jerarquía generalmente se organiza en varios niveles:

- L1 (nivel 1): Más rápida, más pequeña
- L2 (nivel 2): Menos rápida que L1, más grande que L1

- L3 (nivel 3): Más lenta, mayor tamaño. Esta última es compartida por todos los núcleos del procesador.

Estas almacenan datos recientemente utilizados o con alta posibilidad de uso futuro.

Cuando la CPU accede a una dirección de memoria, primero consulta los niveles más cercanos y, en caso de fallo continúa hacia niveles inferiores o memoria principal. Es posible medir la diferencia temporal entre acceso a caché y memoria principal.

Esta diferencia permite que se realice un **ataque de canal lateral,** es decir, un ataque capaz de inferir información de la víctima a través de canales del sistema afectado. De forma que un atacante puede inferir si determinada dirección ha sido accedida previamente, midiendo los tiempos de acceso.

De forma que un atacante podría determinar si una dirección de memoria ha sido accedida previamente. Recordando el funcionamiento de la ejecución especulativa, si una instrucción fue ejecutada de forma anticipada, pese a que fuese descartada por el ROB, los efectos en la caché si pueden permanecer.

Esto es útil en Meltdown y Spectre si durante la ejecución especulativa se accedió a una dirección de memoria dependiente de un dato secreto (contraseñas en memoria, claves API, etc). Esta dirección puede quedar almacenada en caché.

Por lo tanto, un atacante puede medir los tiempos de acceso a distintas posiciones de memoria para identificar cuál de ellas se encuentra en el caché. Dado que la posición accedida depende del valor secreto, es posible inferir indirectamente dicho valor. Este tipo de técnica es conocida como **Flush+Reload**.

Esta técnica comúnmente explota el nivel de caché L3. El atacante vacía repetidamente una ubicación de memoria objetivo utilizando la instrucción clflush. Y mide el tiempo que tarda en cargar los datos.

2. MELTDOWN y SPECTRE

Una vez teniendo el fundamento teórico anterior, podemos dar paso a la explicación de las dos vulnerabilidades abordadas. Para la siguiente sección, dividiremos la explicación, funcionamiento por separado para cada vulnerabilidad, comenzando con MELTDOWN.

2.1 MELTDOWN

Afectando principalmente a procesadores Intel, representa un ataque que permite superar completamente el aislamiento que presenta la memoria virtual, de manera que proporciona una forma sencilla para que cualquier proceso de usuario lea toda la memoria del kernel.

El ataque se aprovecha principalmente de dos vulnerabilidades. Primeramente, aprovecha que cada proceso tiene mapeado en su espacio de memoria virtual una copia del kernel, como mencionamos en la sección 1.1. De forma que un atacante puede ejecutar código en el procesador, obteniendo una copia completa del contenido del espacio de direcciones del kernel.

La segunda vulnerabilidad explotada es el uso de ejecución especulativa, que permite a un proceso acceder a una dirección protegida mientras espera la validación de permisos.

Para este último punto, conviene revisar un ejemplo sencillo en el que se ejemplifica cómo puede ser usada la ejecución especulativa.

```
1 raise_exception();
2 // the line below is never reached
3 access(probe_array[data * 4096]);
```

Nota. Ejemplo de ejecución especulativa. Reading Kernel Memory from User Space (p. 5), por M. Lipp et al., 2018. https://meltdownattack.com/meltdown.pdf

En este ejemplo, es posible entender como la ejecución fuera de orden, y la ejecución especulativa pueden provocar efectos observables en la caché. En el ejemplo, se genera una excepción en la línea 1 (esto podría ser provocada al denegar un acceso a memoria de kernel), por lo tanto, el resto del código no debería de ejecutarse.

Sin embargo, la instrucción de la línea 3 podría ser ejecutada por la ejecución especulativa. Pese a que no se confirman las instrucciones (no dejan efectos en la memoria o registros), sí pueden provocar accesos a memoria que cargan datos en la caché.

El funcionamiento del arreglo "probe_array" lo explicaremos más adelante, pero este es una estructura de datos creada por el atacante en el espacio de memoria de usuario, para obtener lo procesado por el CPU de forma especulativa.

2.1.1 Plan de ataque

Como se comenta en la documentación oficial de Meltdown " *Meltdown: Reading Kernel Memory from User Space*.", el ataque se compone de 3 etapas principales. Estas se basan en el concepto de instrucciones transitivas, las cuales son instrucciones ejecutadas especulativamente o fuera de orden, pero que nunca llegaron a confirmarse debido una excepción o condición inválida como se menciona en el ejemplo del inciso anterior.

Etapa 1: Lectura del secreto.

El atacante hace que la GPU intente leer una dirección de memoria protegida (la del kernel). Aunque se impide el acceso por los bits de privilegio (sección 1.1) y se genere la excepción, la CPU puede llegar a leer el dato por un momento antes de que se determine que no tiene permisos.

Etapa 2: Transmisión del secreto

Dependiendo del valor obtenido, se puede hacer un acceso a memoria por medio de un arreglo utilizado por el atacante (probe_array(data * 4096)). Este acceso provoca que una posición específica del arreglo se cargue en la caché, codificando indirectamente el valor del secreto en el estado de la memoria caché.

Etapa 3: Recepción del secreto.

Por último, el atacante utiliza un ataque de canal lateral (Flush+Reload) para medir los tiempos de acceso a diferentes posiciones del arreglo (sección 1.3). De esta forma es posible identificar en qué posición fue cargada, y por lo tanto, inferir el valor del dato secreto.

2.1.2 Ejemplo de ejecución

A continuación, presentamos la estructura general de código que permite explotar la vulnerabilidad de Meltdown.

```
1. uint8_t* probe_array = new uint8_t[256 * 4096];
2. // ... Make sure probe_array is not cached
3. uint8_t kernel_memory = *(uint8_t*)(kernel_address);
4. uint64_t final_kernel_memory = kernel_memory * 4096;
5. uint8_t dummy = probe_array[final_kernel_memory];
6. // ... catch page fault
7. // ... determine which of 256 slots in probe_array is cached
```

Nota. Estructura general de ataques Meltdown. Meltdown and Spectre, explained. Medium. *por* Klein, M, 2018. https://medium.com/@mattklein123/meltdown-spectre-explained-6bc8634cc0c2

Antes de comenzar con el planteamiento de las 3 etapas abordadas en la ejecución de meltdown, se inicia una etapa de preparación en la línea 1 y 2.

El código crea una estructura de datos (arreglo probe_array) en el espacio de memoria de usuario, Para que el ataque funcione correctamente, en la línea 2, el atacante debe de vaciar continuamente esta ubicación de memoria utilizando la instrucción “clflush”.

Posteriormente, dando comienzo a la etapa 1, en la línea 3 el atacante intenta leer una dirección de memoria protegida del kernel. Recordemos que estás páginas de memoria están protegidas por bits de privilegio, por lo que se producirá un fallo de página. Sin embargo, gracias a la ejecución especulativa, la CPU ejecuta instrucciones antes de que se haya determinado si estas son necesarias y confirmadas, logrando leer el dato momentáneamente.

Continuando con la etapa 2. En la línea 4 y 5, el código toma el dato del kernel y realiza un acceso a memoria por medio del arreglo "probe_array". Al ejecutarse de forma anticipada, esto provoca que una posición específica del arreglo se cargue en la caché, codificando de manera indirecta el valor del secreto en el estado de la memoria caché.

En la línea 6, el procesador evalúa la excepción (por no tener permisos de acceso), y posteriormente, el ROB permite volver a un estado consistente, limpiando y reiniciando las operaciones realizadas. Sin embargo, los efectos en la caché de la lectura especulativa de la línea 5 sí pueden permanecer.

Por último, en la etapa 3, el código utiliza el ataque de canal lateral Flush+Reload, se miden los tiempos de acceso en diferentes posiciones del arreglo, y se determina cuál está en caché y, de esta forma, se infiere el valor del dato secreto.

2.1.3 Medidas de mitigación

Para mitigar la vulnerabilidad que explota Meltdown, se empleó principalmente una solución a nivel software llamado KAISER (KTPI en Linux, KVA Shadow en windows, Double Map en macOS/iOS).

Esta solución implementa un aislamiento más estricto entre el kernel y el espacio de direccionamiento de usuario. Para ello, emplea una tabla de páginas (visto en sección 1,1) para el modo usuario (sin mapeo del kernel) y otra para el modo kernel.

De forma que si un proceso está en modo usuario, el kernel no se encuentra mapeado en el espacio de direcciones accesible, evitando que la ejecución especulativa pueda filtrar datos del kernel a través del caché.

Sin embargo, como consecuencia se tiene un gran impacto en el rendimiento, ya que cada transición entre modo kernel y modo usuario requiere de acciones adicionales para la sincronización de memoria.

2.2 SPECTRE

Spectre es una vulnerabilidad de seguridad de hardware descubierta en 2018 por el equipo de Google Project Zero que afecta a procesadores Intel, AMD, Apple y ARM.

Esta vulnerabilidad rompe el aislamiento entre diferentes aplicaciones, lo que permite a un atacante leer información privada (datos sensibles) de la memoria a otros programas, a pesar de ser una vulnerabilidad más difícil de explotar que Meltdown es igual de difícil de mitigar, ya que se trata de una vulnerabilidad a nivel de la arquitectura del procesador.

2.2.1 Plan de ataque

Spectre funciona con ayuda de la ejecución especulativa (Sección 1.2.1), ya que estos ataques consisten en inducir la realización de operaciones de forma especulativa que no ocurrirían durante la ejecución serializada de las instrucciones del programa.

Fase de configuración:
En la mayoría de los casos el ataque comienza con una fase de configuración, en la que el atacante realiza operaciones que desentrenan al procesador para que luego haga una predicción especulativa errónea que pueda ser explotada. Además, la fase de configuración suele incluir pasos que ayudan a inducir la ejecución especulativa, como manipular el estado de la caché para eliminar datos que el procesador necesitará para determinar el flujo de control real.

Fase de preparación:
Durante la fase de preparación, el atacante puede preparar el canal encubierto que se utilizará para extraer la información de la víctima, por ejemplo, realizando parte de vaciado o expulsión de un ataque Flush+Reload (Sección 1.3).

Fase de ejecución:
En la fase de ejecución el procesador ejecuta de forma especulativa una o varias instrucciones que transfieren información confidencial desde el contexto de la víctima a un canal encubierto. Esto puede desencadenarse cuando el atacante solicita a la víctima que realice una acción, por ejemplo, a través de una llamada al sistema, un socket o un archivo. En otros casos, el atacante puede aprovechar la ejecución especulativa (errónea) de su propio código para obtener información confidencial del mismo proceso. Por ejemplo, el código de ataque que está aislado en un entorno de pruebas por un intérprete, un compilador justo a tiempo o un lenguaje “seguro” podría intentar leer memoria a la que no se supone que debe acceder.

Fase de recuperación de datos:
El proceso de recuperación consiste en sincronizar el acceso a las direcciones de memoria en las líneas de caché que se están monitoreando. Los ataques Spectre solo asumen que las instrucciones ejecutadas de forma especulativa pueden leerse desde la memoria a la que el proceso de la víctima podría acceder normalmente.

2.2.2 Ejemplo de ejecución

Variante del aprovechamiento del error de predicciones en las ramificaciones condicionales

En esta variante consideramos un código, como el siguiente:

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```

Nota. Ejemplo de acceso especulativo en Spectre. Exploiting speculative execution (p. 2), por Kocher et al., 2018, https://spectreattack.com/spectre.pdf

Considerando que forma parte de una función de una llamada al sistema o una biblioteca, que recibe un entero x de una fuente confiable. El proceso que ejecuta el código tiene acceso a una matriz de bytes array1 de tamaño array1_size, y a una segunda matriz de bytes array2 de tamaño 1 MB. La condicional es una comprobación la cual evita que el procesador lea memoria confidencial fuera de array1.

Aquí es donde comienza la predicción especulativa, ya que antes de que se conozca el resultado correcto de la comprobación de límites, el predictor continúa con el destino de ramificación más probable, lo que conduce a una aceleración general de la ejecución si el resultado se predijo correctamente. Sin embargo, si la comprobación de límites se predice incorrectamente como verdadera, un atacante puede filtrar información confidencial.

Durante la predicción especulativa, la rama condicional que verifica los límites de "X" puede seguir una ruta incorrecta. Esto es lo que hace el atacante, ya que hace que X esté fuera de los límites de array1 haciendo que este se resuelva en en un byte secreto "k" en algún lugar de la memoria, lo que hace que k esté guardado en el caché. Las operaciones anteriores recibieron valores de x que eran válidos, lo que llevó al predictor a suponer que el if probablemente sería verdadero. Esta configuración de caché puede darse de forma natural o puede ser creada por un atacante, por ejemplo, provocando la expulsión de array1_size y array2 y haciendo que el núcleo utilice la clave secreta en una operación legítima.

2.2.3 Medidas de mitigación

Algunas propuestas de mitigación para esta vulnerabilidad abordan una o varias características en las que se basan los ataques.

La prevención de la predicción especulativa pese a ser la solución más sencilla no es viable, ya que evitar la ejecución especulativa provocaría una degradación significativa del rendimiento del procesador. Aunque los procesadores actuales no parecen disponer de métodos que permitan al software desactivar la ejecución especulativa, dichos modos podrían añadirse en futuros procesadores o, en algunos casos, podrían introducirse mediante cambios en el microcódigo.

Evitar que los datos entren en canales ocultos es otra opción que propone que en futuros procesadores se podrían detectar si los datos se han obtenido como resultado de una

operación especulativa, y en caso de que así sea, impedir que dichos datos se utilicen en operaciones posteriores.

Existen gran cantidad de medidas para mitigar esta vulnerabilidad, sin embargo al tratarse de una vulnerabilidad de hardware es muy poco probable que pueda ser resuelta en un futuro próximo, ya que se necesitaría cambiar la arquitectura de los procesadores creados hasta ahora.

3 Conclusión final

En conclusión, por medio de las vulnerabilidades de Meltdown y Spectre se pudo demostrar que ciertas técnicas de optimización utilizadas en los procesadores modernos, como pueden ser la ejecución especulativa y la ejecución fuera de orden. pueden producir riesgos importantes de seguridad. Si bien estas técnicas tenían el propósito de mejorar el rendimiento, se demostró que pueden ser utilizadas para acceder de forma indirecta a información que debería permanecer protegida.

En general, Meltdown muestra cómo es posible burlar el aislamiento entre espacio de usuario y el kernel mediante el uso de efectos en la memoria caché. Por otro lado, Spectre muestra que el propio flujo de ejecución de un programa, puede ser manipulado para filtrar información.

El impacto de estas vulnerabilidades fue considerable, ya que afectaron a una gran variedad de dispositivos, lo que obligó a desarrollar parches y mecanismos de mitigación que, en algunos casos, implicaron a una reducción en el rendimiento de los sistemas.

Bibliografía

[1]Klein, M. (2018, enero 15). *Meltdown and Spectre, explained*. Medium. https://medium.com/@mattklein123/meltdown-spectre-explained-6bc8634cc0c2

[2] Moniente, M. (2019). *Análisis de vulnerabilidades de hardware basadas en la ejecución especulativa.* [Trabajo Fin de Grado, Universidad de Zaragoza]. https://zaguan.unizar.es/record/84976

[3] Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Fogh, A., Horn, J., Mangard, S., Kocher, P., Genkin, D., Yarom, Y., y Hamburg, M. (2018). *Meltdown: Reading Kernel Memory from User Space*. https://meltdownattack.com/meltdown.pdf

[4] Spectre Attack. (s. f.). Meltdown and Spectre. https://spectreattack.com

[5] Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., & Yarom, Y. (2018). *Spectre attacks: Exploiting speculative execution*. https://spectreattack.com/spectre.pdf

[6]Red Hat. (2018, enero 5). ¿Qué son Spectre y Meltdown? Esto es lo que necesita saber. https://www.redhat.com/es/blog/%C2%BFqu%C3%A9-son-spectre-y-meltdown-esto-es-lo-que-necesita-saber